Lab Sheet 5: SQL Joins

Complete SQL code with sample tables, queries, and expected outputs

Dialect note: The main examples use MySQL 8.0 syntax. The FULL OUTER JOIN example uses a UNION workaround because MySQL does not directly support FULL OUTER JOIN. Outputs below are based on the sample data created in this document.

1. Create and populate sample tables

```
CREATE DATABASE IF NOT EXISTS JoinLab;
USE JoinLab;

CREATE TABLE Department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL
);

CREATE TABLE Course (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(50) NOT NULL,
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);

CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(50) NOT NULL,
    dept_id INT NULL,
    course_id INT NULL,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id),
    FOREIGN KEY (course_id) REFERENCES Course(course_id)
);

CREATE TABLE Employee (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50) NOT NULL,
    dept_id INT NULL,
    manager_id INT NULL,
    salary DECIMAL(10,2),
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id),
    FOREIGN KEY (manager_id) REFERENCES Employee(emp_id)
);

INSERT INTO Department VALUES
(10, 'Computer Science'),
(20, 'Business'),
(30, 'Mechanical'),
(40, 'Civil');

INSERT INTO Course VALUES
(101, 'Database Systems', 10),
(102, 'Web Development', 10),
(201, 'Marketing', 20),
(301, 'Thermodynamics', 30);

INSERT INTO Student VALUES
(1, 'Aarav', 10, 101),
(2, 'Diya', 10, 102),
(3, 'Kabir', 20, 201),
(4, 'Meera', NULL, NULL),
(5, 'Rohan', 30, 301);

INSERT INTO Employee VALUES
(1, 'Anita', 10, NULL, 90000),
(2, 'Bharat', 10, 1, 60000),
(3, 'Charu', 20, NULL, 70000),
(4, 'Deepak', 20, 3, 50000),
(5, 'Esha', 30, 1, 55000);
```

Sample data note: Department 40 (Civil) has no students; Meera has no department/course; course 301 has Rohan enrolled. This lets the outer joins show unmatched rows.

2. Queries and expected outputs

Run each numbered query after executing the setup script. NULL means the matching value is missing. Output spacing is illustrative; database clients may format columns differently.

1. INNER JOIN between Student and Department

SQL:

```
SELECT s.student_id, s.student_name, d.dept_name
FROM Student AS s
INNER JOIN Department AS d ON s.dept_id = d.dept_id;
```

Expected output:

```
student_id  student_name  dept_name
1           Aarav         Computer Science
2           Diya          Computer Science
3           Kabir         Business
5           Rohan         Mechanical
```

2. Student names with department names (INNER JOIN)

SQL:

```
SELECT s.student_name, d.dept_name
FROM Student s
INNER JOIN Department d ON s.dept_id = d.dept_id
ORDER BY s.student_id;
```

Expected output:

```
student_name  dept_name
Aarav          Computer Science
Diya           Computer Science
Kabir          Business
Rohan          Mechanical
```

3. Employee names with department names

SQL:

```
SELECT e.emp_name, d.dept_name
FROM Employee e
INNER JOIN Department d ON e.dept_id = d.dept_id
ORDER BY e.emp_id;
```

Expected output:

```
emp_name  dept_name
Anita     Computer Science
Bharat    Computer Science
Charu     Business
Deepak    Business
Esha      Mechanical
```

4. Students with course details

SQL:

```
SELECT s.student_name, c.course_name
FROM Student s
INNER JOIN Course c ON s.course_id = c.course_id
ORDER BY s.student_id;
```

Expected output:

```
student_name  course_name
Aarav          Database Systems
Diya           Web Development
Kabir          Marketing
Rohan          Thermodynamics
```

5. Students in a particular department (Computer Science)

SQL:

```
SELECT s.student_name, d.dept_name
FROM Student s
INNER JOIN Department d ON s.dept_id = d.dept_id
WHERE d.dept_name = 'Computer Science';
```

Expected output:

```
student_name  dept_name
Aarav          Computer Science
Diya           Computer Science
```

6. LEFT OUTER JOIN between Student and Department

SQL:

```
SELECT s.student_name, d.dept_name
FROM Student s
LEFT OUTER JOIN Department d ON s.dept_id = d.dept_id
ORDER BY s.student_id;
```

Expected output:

```
student_name  dept_name
Aarav          Computer Science
Diya           Computer Science
Kabir          Business
Meera          NULL
Rohan          Mechanical
```

7. All students, including students without a matching department

SQL:

```
SELECT s.student_name, d.dept_name
FROM Student s
LEFT JOIN Department d ON s.dept_id = d.dept_id
ORDER BY s.student_id;
```

Expected output:

```
student_name  dept_name
Aarav          Computer Science
Diya           Computer Science
Kabir          Business
Meera          NULL
Rohan          Mechanical
```

8. RIGHT OUTER JOIN between Student and Department

SQL:

```
SELECT s.student_name, d.dept_name
FROM Student s
RIGHT OUTER JOIN Department d ON s.dept_id = d.dept_id
ORDER BY d.dept_id, s.student_id;
```

Expected output:

```
student_name  dept_name
Aarav          Computer Science
Diya           Computer Science
Kabir          Business
Rohan          Mechanical
NULL           Civil
```

9. All departments, including departments with no students

SQL:

```
SELECT d.dept_name, s.student_name
FROM Student s
RIGHT JOIN Department d ON s.dept_id = d.dept_id
ORDER BY d.dept_id, s.student_id;
```

Expected output:

```
dept_name         student_name
Computer Science  Aarav
Computer Science  Diya
Business          Kabir
Mechanical        Rohan
Civil             NULL
```

10. Students not assigned to any department using LEFT JOIN

SQL:

```
SELECT s.student_id, s.student_name
FROM Student s
LEFT JOIN Department d ON s.dept_id = d.dept_id
WHERE d.dept_id IS NULL;
```

Expected output:

```
student_id  student_name
4           Meera
```

11. Departments having no students using RIGHT JOIN

SQL:

```
SELECT d.dept_id, d.dept_name
FROM Student s
RIGHT JOIN Department d ON s.dept_id = d.dept_id
WHERE s.student_id IS NULL;
```

Expected output:

```
dept_id  dept_name
40       Civil
```

12. FULL OUTER JOIN concept (MySQL UNION workaround)

SQL:

```
SELECT s.student_name, d.dept_name
FROM Student s
LEFT JOIN Department d ON s.dept_id = d.dept_id
UNION
SELECT s.student_name, d.dept_name
FROM Student s
RIGHT JOIN Department d ON s.dept_id = d.dept_id
ORDER BY dept_name, student_name;
```

Expected output:

```
student_name  dept_name
Kabir          Business
NULL           Civil
Aarav          Computer Science
Diya           Computer Science
Meera          NULL
Rohan          Mechanical
```

13. CROSS JOIN between Student and Course

SQL:

```
SELECT s.student_name, c.course_name
FROM Student s
CROSS JOIN Course c
ORDER BY s.student_id, c.course_id;
```

Expected output:

```
This returns every student paired with every course.
There are 5 students x 4 courses = 20 rows.
First rows:
Aarav  Database Systems
Aarav  Web Development
Aarav  Marketing
Aarav  Thermodynamics
Diya   Database Systems
... (remaining rows follow the same pattern)
```

14. Number of possible Student-Course combinations

SQL:

```
SELECT COUNT(*) AS total_combinations
FROM Student
CROSS JOIN Course;
```

Expected output:

```
total_combinations
20
```

15. NATURAL JOIN between Student and Department

SQL:

```
-- Both tables share the column name dept_id.
SELECT student_name, dept_name
FROM Student
NATURAL JOIN Department
ORDER BY student_id;
```

Expected output:

```
student_name  dept_name
Aarav          Computer Science
Diya           Computer Science
Kabir          Business
Rohan          Mechanical
```

16. Compare NATURAL JOIN and INNER JOIN

SQL:

```
-- NATURAL JOIN automatically matches all same-named columns.
SELECT student_name, dept_name
FROM Student NATURAL JOIN Department;

-- Explicit equivalent for this schema:
SELECT s.student_name, d.dept_name
FROM Student s
INNER JOIN Department d ON s.dept_id = d.dept_id;
```

Expected output:

```
Both queries return the same four matched students here.
INNER JOIN is usually clearer and safer because the join condition is explicit.
NATURAL JOIN can change behavior if another same-named column is later added.
```

17. SELF JOIN on Employee for employee-manager relationships

SQL:

```
SELECT e.emp_name AS employee, m.emp_name AS manager
FROM Employee e
LEFT JOIN Employee m ON e.manager_id = m.emp_id
ORDER BY e.emp_id;
```

Expected output:

```
employee  manager
Anita     NULL
Bharat    Anita
Charu     NULL
Deepak    Charu
Esha      Anita
```

18. Employees along with their managers

SQL:

```
SELECT e.emp_name AS employee_name,
       COALESCE(m.emp_name, 'No manager') AS manager_name
FROM Employee e
LEFT JOIN Employee m ON e.manager_id = m.emp_id
ORDER BY e.emp_id;
```

Expected output:

```
employee_name  manager_name
Anita          No manager
Bharat         Anita
Charu          No manager
Deepak         Charu
Esha           Anita
```

19. Join three tables: Student, Department and Course

SQL:

```
SELECT s.student_name, d.dept_name, c.course_name
FROM Student s
LEFT JOIN Department d ON s.dept_id = d.dept_id
LEFT JOIN Course c ON s.course_id = c.course_id
ORDER BY s.student_id;
```

Expected output:

```
student_name  dept_name         course_name
Aarav          Computer Science  Database Systems
Diya           Computer Science  Web Development
Kabir          Business          Marketing
Meera          NULL              NULL
Rohan          Mechanical        Thermodynamics
```

20. Student name, course name and department name

SQL:

```
SELECT s.student_name, c.course_name, d.dept_name
FROM Student s
LEFT JOIN Course c ON s.course_id = c.course_id
LEFT JOIN Department d ON s.dept_id = d.dept_id
ORDER BY s.student_id;
```

Expected output:

```
student_name  course_name        dept_name
Aarav          Database Systems    Computer Science
Diya           Web Development     Computer Science
Kabir          Marketing           Business
Meera          NULL                NULL
Rohan          Thermodynamics      Mechanical
```

21. Employees earning more than their department's average salary

SQL:

```
SELECT e.emp_name, d.dept_name, e.salary
FROM Employee e
JOIN Department d ON e.dept_id = d.dept_id
JOIN (
    SELECT dept_id, AVG(salary) AS avg_salary
    FROM Employee
    WHERE dept_id IS NOT NULL
    GROUP BY dept_id
) a ON e.dept_id = a.dept_id
WHERE e.salary > a.avg_salary
ORDER BY e.emp_id;
```

Expected output:

```
emp_name  dept_name         salary
Anita     Computer Science  90000.00
Charu     Business          70000.00
Esha      Mechanical        55000.00
```

22. Department with the maximum number of students

SQL:

```
SELECT d.dept_name, COUNT(s.student_id) AS student_count
FROM Department d
LEFT JOIN Student s ON s.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name
HAVING COUNT(s.student_id) = (
    SELECT MAX(student_count)
    FROM (
        SELECT COUNT(s2.student_id) AS student_count
        FROM Department d2
        LEFT JOIN Student s2 ON s2.dept_id = d2.dept_id
        GROUP BY d2.dept_id
    ) AS counts_by_dept
);
```

Expected output:

```
dept_name         student_count
Computer Science  2
```

23. Departments and total number of students in each

SQL:

```
SELECT d.dept_name, COUNT(s.student_id) AS total_students
FROM Department d
LEFT JOIN Student s ON s.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name
ORDER BY d.dept_id;
```

Expected output:

```
dept_name         total_students
Computer Science  2
Business          1
Mechanical        1
Civil             0
```

24. Courses with number of students enrolled in each course

SQL:

```
SELECT c.course_name, COUNT(s.student_id) AS enrolled_students
FROM Course c
LEFT JOIN Student s ON s.course_id = c.course_id
GROUP BY c.course_id, c.course_name
ORDER BY c.course_id;
```

Expected output:

```
course_name       enrolled_students
Database Systems  1
Web Development   1
Marketing         1
Thermodynamics    1
```

25. Student-department-course report using INNER JOIN, LEFT JOIN, GROUP BY and aggregates

SQL:

```
SELECT d.dept_name,
       c.course_name,
       COUNT(s.student_id) AS total_students,
       COUNT(DISTINCT s.student_id) AS unique_students
FROM Department d
LEFT JOIN Student s ON s.dept_id = d.dept_id
LEFT JOIN Course c ON c.course_id = s.course_id
GROUP BY d.dept_id, d.dept_name, c.course_id, c.course_name
ORDER BY d.dept_id, c.course_id;
```

Expected output:

```
dept_name         course_name        total_students  unique_students
Computer Science  Database Systems    1               1
Computer Science  Web Development     1               1
Business          Marketing           1               1
Mechanical        Thermodynamics      1               1
Civil             NULL                0               0
```

Quick revision: join types

Join type	What it returns
INNER JOIN	Only rows that match in both tables.
LEFT JOIN	Every row from the left table plus matching right-table rows; unmatched right columns are NULL.
RIGHT JOIN	Every row from the right table plus matching left-table rows; unmatched left columns are NULL.
FULL OUTER JOIN	All matched and unmatched rows from both sides; use UNION of LEFT and RIGHT joins in MySQL.
CROSS JOIN	Every possible combination of rows from both tables.
NATURAL JOIN	Automatically joins using every same-named column in both tables.
SELF JOIN	Joins a table to itself, often to represent manager relationships.

Important: In Question 25, the report uses LEFT JOINs so departments with no students remain visible. If your instructor requires an INNER JOIN explicitly in the final report, replace the first LEFT JOIN with INNER JOIN; doing so will omit departments with no students.